

BabyShopHub

Developer Guide

Aptech eProject Submission

Version 1.0

Table of Contents

1. Who this guide is for.....	1
2. Building and running.....	3
3. Project structure.....	4
3.1 The barrels.....	4
4. Layer responsibilities in code.....	5
5. Conventions, and why they were chosen.....	6
5.1 Result-or-message returns.....	6
5.2 Single sources of truth.....	6
5.3 API-stability choices.....	6
6. Data and demo accounts.....	7
7. How to make common changes.....	8
7.1 Add a product (in code / seed).....	8
7.2 Add a screen.....	8
7.3 Add a repository method.....	8
7.4 Change money or tax rules.....	8
8. Verifying the code.....	9
9. Known limitations.....	10

1. Who this guide is for

This guide orients a developer in the BabyShopHub codebase: how it is laid out, how to build and run it, the conventions it follows and why, and how to make the common kinds of

change. It assumes you have read the System Design document for the architectural picture; this guide is the practical, code-level companion.

The project is **72 Dart files, about 16,300 lines**, all under lib/, plus a tool/ folder of helper scripts and an assets/ folder of images.

2. Building and running

Full, from-nothing Windows instructions are in the *Windows Setup Guide*. In short, from the project root:

```
flutter create .    # regenerate platform folders (first time only)
flutter pub get     # fetch dependencies
flutter run -d windows # run as a desktop window (easiest to demo)
```

The project ships as source only — the android/, windows/, and other platform folders are generated on your machine by `flutter create .` so they match your Flutter version. That command does not touch lib/, pubspec.yaml, or assets/.

To check the code without running it:

```
flutter analyze     # static analysis; should report no issues
```

3. Project structure

Everything is under `lib/`, grouped by responsibility:

Folder	Contents
<code>lib/core/</code>	Cross-cutting constants and helpers: <code>app_config</code> , <code>theme</code> , <code>pricing</code> , <code>validators</code> , <code>statuses</code> , <code>formats</code> , <code>db_utils</code> .
<code>lib/models/</code>	Plain data classes (11): <code>AppUser</code> , <code>Product</code> , <code>ProductCategory</code> , <code>Seller</code> , <code>CartItem</code> , <code>Order</code> , <code>Address</code> , <code>PaymentMethod</code> , <code>Review</code> , <code>SupportTicket</code> , plus the <code>models.dart</code> barrel.
<code>lib/data/</code>	The database (<code>database_helper</code>), the seed data, <code>password_service</code> , <code>faq_data</code> , and seven repositories.
<code>lib/providers/</code>	<code>SessionProvider</code> , <code>CartProvider</code> , and the <code>providers.dart</code> barrel.
<code>lib/screens/</code>	The 35 screens: <code>auth/</code> (2), <code>customer/</code> (21 incl. the shell), <code>admin/</code> (11), and the top-level <code>splash_screen</code> .
<code>lib/widgets/</code>	Reusable UI (<code>common</code> , <code>product_card</code> , <code>star_rating</code>) and the <code>widgets.dart</code> barrel.
<code>lib/main.dart</code>	App entry, provider wiring, and the root routing gate.
<code>tool/</code>	Developer helper scripts, run manually and not part of the shipped app: <code>generate_images.py</code> (regenerates the placeholder product images) and <code>check_project.py</code> (a quick structural sanity check).

3.1 The barrels

`models.dart`, `providers.dart`, and `widgets.dart` are **barrel** files that re-export their folder's public classes so a screen can write one import instead of many. Note that the barrels do **not** re-export each other: a screen that uses `AppUser` (a model) must import `models.dart` explicitly even if it already imports `providers.dart`. This is deliberate — it keeps the dependency between layers visible rather than hidden behind a chain of re-exports.

4. Layer responsibilities in code

The layers described in the System Design document map directly to folders:

Screens (`lib/screens`) build the UI and call repositories or watch providers. They never open the database. A typical screen loads its data in a `Future` and renders it through the shared `AsyncView` widget, which handles the loading and error states in one place.

Providers (`lib/providers`) are `ChangeNotifiers` for the two pieces of shared state — the session and the cart. Screens read them with `context.watch` (to rebuild on change) or `context.read` (to call a method).

Repositories (`lib/data/*_repository.dart`) are the only classes that run SQL. Each groups the operations for one area (auth, catalogue, cart, orders, reviews, support, admin). They convert rows to models and back.

DatabaseHelper (`lib/data/database_helper.dart`) owns the single connection, creates the schema and indexes on first launch, seeds the sample data, and exposes the rating-recomputation helpers.

5. Conventions, and why they were chosen

This project makes some deliberate choices for stability and clarity. Keep to them when editing.

5.1 Result-or-message returns

Repository **write** methods and all **validators** return `String?`: null means success, and a non-null value is a human-readable error message ready to show the user. This is why screens can do:

```
final String? error = await repo.doThing(...);  
if (error != null) { showSnack(context, error, error: true); return; }
```

It avoids exceptions for ordinary validation failures and puts the wording of an error next to the rule that produced it.

5.2 Single sources of truth

- **Pricing** lives only in `lib/core/pricing.dart`. Checkout, the cart summary, and the seeded orders all call it, so a receipt can never disagree with the cart. Change the tax rate or delivery fee in `app_config.dart` and everything follows.
- **Statuses** (order and ticket) live only in `lib/core/statuses.dart`, including the allowed transitions. The admin UI reads `nextOptions` rather than hard-coding buttons.
- **Theme** (colours, spacing, radii, text styles, input and button styles) lives only in `lib/core/theme.dart`. Screens reference `AppColors`, `AppSpacing`, etc.; no screen invents its own colour.

5.3 API-stability choices

To keep the build robust across Flutter versions, the code intentionally avoids some newer or more fragile APIs:

- Material 3 is on, but no component-theme classes are used.
- No Dart 3 record types; small groupings use named classes instead.
- `DropDownButtonFormField` uses `value:` (not the newer `initialValue:`).
- Switches use `activeColor` (not `activeThumbColor`).
- Colour opacity uses `color.withValues(alpha:)`.

If you edit a screen, matching these keeps flutter analyze clean on the widest range of SDK versions.

6. Data and demo accounts

On first launch, DatabaseHelper creates the schema and loads the sample data from `lib/data/seed_data.dart` inside a single transaction. The seed contains:

Data

Users (1 admin + 4 customers)

Categories

Sellers

Products

Product reviews

Seller ratings

Saved addresses

Demo orders

Support tickets

Support messages

The two accounts used for demonstration:

Role

Customer

Administrator

DatabaseHelper also exposes a `resetDatabase()` for demos, and the schema is recreated automatically if the database file is deleted.

7. How to make common changes

7.1 Add a product (in code / seed)

Products are normally added through the **admin Product form** at runtime. To add one to the seeded data, add a map to the products list in `seed_data.dart` with a name, brand, price, category_id, seller_id, stock, and an image_path, and drop the matching image into `assets/images/products/`. Image paths follow `assets/images/products/<stem>.png`.

7.2 Add a screen

1. Create the screen file under the right `lib/screens/` subfolder.
2. Import the barrels you need (`models.dart`, `providers.dart`, `widgets.dart`) plus any repository.
3. Reach it with a `MaterialPageRoute`, or add it as a tab in the relevant shell (`customer_shell.dart` or `admin_shell.dart`).
4. Use `AppColors/AppSpacing/etc.` from the theme and the shared widgets so it matches the rest of the app.

7.3 Add a repository method

1. Add the method to the appropriate repository in `lib/data/`.
2. Return `Future<String?>` for a write (null on success, message on failure), or `Future<T>` for a read.
3. Do all SQL through the injected Database; do not open a second connection.
4. If it changes reviews, call the rating-recompute helper so totals stay correct.

7.4 Change money or tax rules

Edit the constants in `lib/core/app_config.dart` (delivery fee, free-delivery threshold, tax rate, currency symbol). Do not scatter these values into screens — Pricing reads them centrally.

8. Verifying the code

The authoritative check on the codebase is the Dart analyzer:

```
flutter analyze
```

It type-checks every file, resolves every reference, and applies the lint rules configured in `analysis_options.yaml` (the standard `flutter_lints` set). Run it from the project root after any change; on this project it reports No issues found!. Because the analyzer understands the whole project, a clean result means imports resolve, types line up, and no referenced widget, method, or asset is missing.

Run `flutter analyze` first, and then `flutter run` (see the Setup Guide) to launch the app and confirm behaviour. Together these are all the verification the project needs.

9. Known limitations

These are deliberate scope boundaries for the eProject, not defects:

- **No real payment.** Checkout is simulated; no gateway is integrated and no real charge occurs.
- **No network or cloud.** All data is local to the device. There is no account server, so there is no cross-device sync and no email-based password reset — an administrator resets a password directly.
- **Single local database.** Uninstalling the app removes all data.
- **Not formally accessibility-certified.** The UI is built to be accessible (contrast, tap targets, labels) but has not been validated with assistive technologies.
- **Seed images are placeholders.** Product images are generated placeholders suitable for demonstration.

For the list of test cases that exercise the behaviours described here, see the Test Plan.